SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0612	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	**JAYAKRISHNA.V**	**Reg. No.:**	**192421259**
Section / Batch:	**I**	**Date:**	**28.07.26**

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q29

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I (JAYAKRISHNA.V/ 192421259) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JAYAKRISHNA.V

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q33. Debugging Maximum–Minimum Finder Code (Improper Initialization for Negative Inputs)

1. Problem Analysis

The given Maximum–Minimum Finder code is:

```
def min_max(arr):
    mn = 0
    mx = 0
    for x in arr:
        if x < mn:
            mn = x
        if x > mx:
            mx = x
    return mn, mx
```

Problem

The function attempts to find the minimum and maximum values in an array using the Brute Force technique.
However, the variables **mn** and **mx** are initialized to **0** before traversing the array.
Since **0** may not be present in the dataset, the algorithm fails for arrays containing only positive numbers or only negative numbers.
As a result, the program returns incorrect minimum or maximum values.

Example

Array = [-8, -3, -15, -6]

The function updates **mn** and **mx** as follows:

Step	Index	Element	pos Value
Initial	-	0	0
1	-8	-8	0

2	-3	-8	0
3	-15	-15	0
4	-6	-15	0

Returned value:

(-15, 0)

But the expected answer is:

(-15, -3)

2. Why the Intended Search Behavior is Violated

The statements
mn = 0
mx = 0

assume that **0** is a valid starting value for finding both the minimum and maximum.
Since there is no guarantee that **0** exists in the given array, incorrect comparisons occur.
For an array containing only negative numbers, **mx** never changes because no element is greater than **0**.
For an array containing only positive numbers, **mn** never changes because no element is smaller than **0**.
Therefore, the algorithm returns incorrect minimum or maximum values.

3. Corrected Solution

The minimum and maximum values should be initialized using the first element of the array.

```
def min_max(arr):
    mn = arr[0]
    mx = arr[0]
    for x in arr:

        if x < mn:
            mn = x
        if x > mx:
            mx = x

    return mn, mx
```

4. Optimized Version (Pythonic)

```
def min_max(arr):
    minimum = maximum = arr[0]
    for value in arr[1:]:
        if value < minimum:
            minimum = value
        if value > maximum:
            maximum = value
    return minimum, maximum
```

This version is more readable because meaningful variable names are used and the first element is already considered before the loop begins.

5. Sample Execution

Input

Array = [-8, -3, -15, -6]

Output

Minimum value = -15
Maximum value = -3

6. Time Complexity Analysis

Let **n** be the number of elements.

Case	**Explanation**	**Time Complexity**
Best Case	Every element is visited once	**O(n)**

Average Case	All elements are compared once	**O(n)**
Worst Case	Entire array is traversed	**O(n)**

Space Complexity

Only a few variables are used.

Auxiliary Space = O(1)

7. Advantages of the Corrected Code

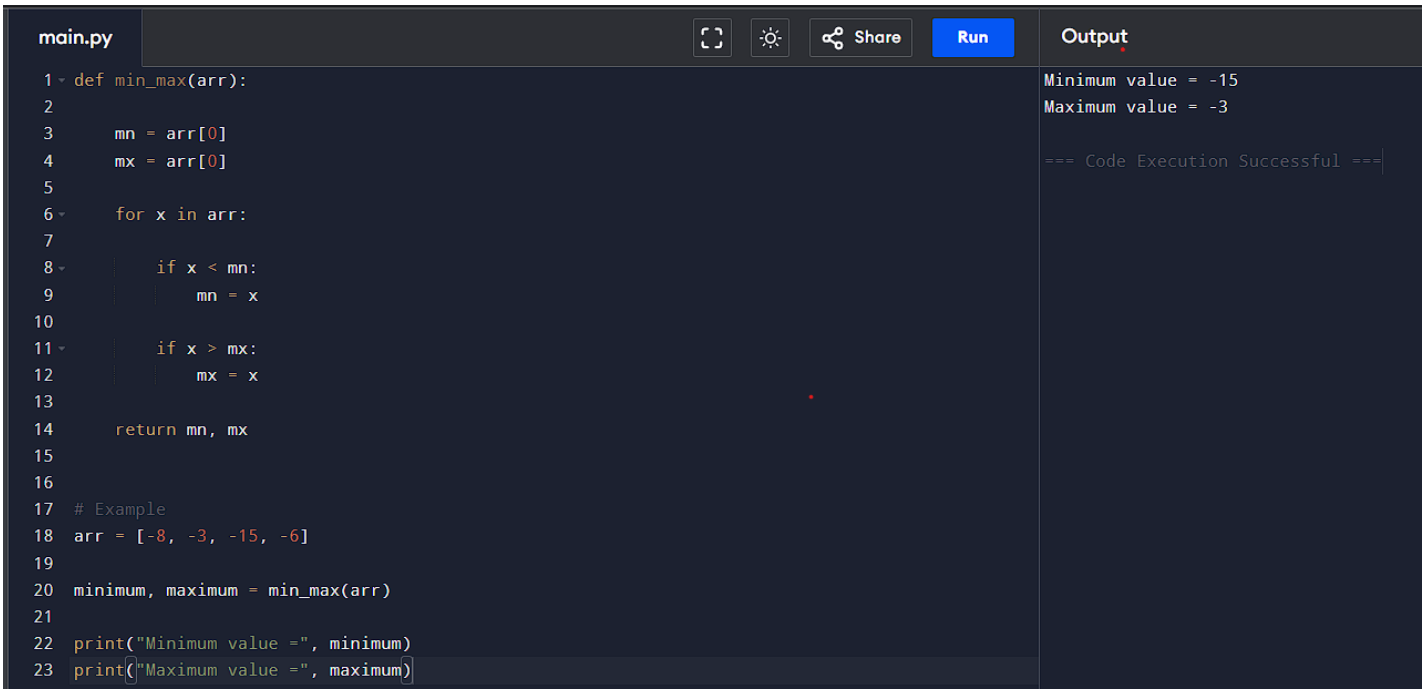
- Correctly handles positive arrays.
- Correctly handles negative arrays.
- Works for mixed datasets.
- Traverses the array only once.
- Improves readability and correctness.
- Uses constant extra memory **O(1)**.

8. Final Correct Code (with Comments)

```
def min_max(arr):

    # Initialize minimum and maximum
    # with the first array element
    mn = arr[0]
    mx = arr[0]

    # Traverse the array
    for x in arr:

        # Update minimum value
        if x < mn:
            mn = x

        # Update maximum value
        if x > mx:
            mx = x

    # Return minimum and maximum values
    return mn, mx
```

```
# Example
arr = [-8, -3, -15, -6]

minimum, maximum = min_max(arr)

print("Minimum value =", minimum)
print("Maximum value =", maximum)
```

Output



Conclusion

The original program failed because both the minimum and maximum variables were initialized to **0**, which is not suitable for all datasets. As a result, incorrect values were returned for arrays containing only positive or only negative numbers. The corrected implementation initializes both variables with the **first element of the array**, ensuring accurate comparisons throughout the traversal. The algorithm correctly handles positive, negative, and mixed datasets while preserving the Brute Force approach. The corrected solution has a time complexity of **O(n)** and an auxiliary space complexity of **O(1)**, making it simple, efficient, and reliable.